Universidad San Carlos de Guatemala
Facultad de ingeniería.
Ingeniería en ciencias y sistemas

Título del Proyecto:

Bank usac

PONDERACIÓN: 14 pts

Tiempo estimado: 50 hrs

Índice

Contenido

1. MARCO FORMATIVO 2
1.1. Valor 2
1.2. Competencia(s) 2
1.3. Habilidad(es) blandas a formar 2
2. Resultado del Aprendizaje 3
2.1. Objetivo SMART 3
3. Resumen Ejecutivo 3
4. Enunciado del Proyecto 3
4.1 Descripción del problema o necesidad a resolver 3
4.2 Alcance del proyecto 7
Flexibilidad de diseño 13
4.3 Recursos y herramientas a utilizar 13
4.4 Entregables 14
5. Material de apoyo 14
6. Metodología 16
7. Cronograma 18
8. Rúbrica de Calificación 19
8.1 Requisitos para optar a la calificación 19
8.2 Detalle de la Calificación 21
8.3 Comentarios Generales 26

1. MARCO FORMATIVO

1.1. Valor

Nombre del valor	¿Cómo se aplica en el proyecto?
Responsabilidad compartida (Ownership)	Cada integrante del equipo asume la responsabilidad completa de su microservicio, no solo en desarrollo sino también en su confiabilidad, disponibilidad y comportamiento ante fallos. Este enfoque refleja la cultura DevOps y el rol de SRE, donde los equipos son dueños del ciclo de vida completo del servicio, incluyendo diseño, implementación y resiliencia.

1.2. Competencia(s)

Diseñar e implementar un sistema distribuido basado en microservicios, aplicando principios de **DevOps y SRE**, garantizando disponibilidad, resiliencia y escalabilidad en un entorno Kubernetes.

1.3. Habilidad(es) blandas a formar

- Trabajo colaborativo (equipo de 3)
- Comunicación técnica
- Toma de decisiones basada en trade-offs
- Pensamiento sistémico
- Ownership de servicios (mentalidad SRE)

2. Resultado del Aprendizaje

2.1. Objetivo SMART

Específico (¿Qué?)	**Medible (¿Cuánto?)**	**Alcanzable (¿Cómo?)**	**Relevante (¿Para qué?)**	**A Tiempo (¿Cuándo?)**
Diseñar sistema bancario distribuido	5 microservicios funcionales	Uso de K8s local + tecnologías definidas	Simulación de banca.	4 semanas

3. Resumen Ejecutivo

Este proyecto consiste en el diseño de un sistema bancario basado en microservicios, desplegado en un cluster local de Kubernetes, aplicando principios de DevOps y SRE desde su concepción.

El sistema estará compuesto por 5 microservicios independientes que se comunicarán de forma asincrónica mediante un broker de mensajería. Se definirán métricas de confiabilidad (SLI), objetivos (SLO) y acuerdos (SLA), además de documentación completa de arquitectura.

4. Enunciado del Proyecto

4.1 Descripción del problema o necesidad a resolver

Las instituciones financieras requieren sistemas altamente disponibles, escalables y resilientes capaces de procesar transacciones de manera confiable en entornos distribuidos. Los sistemas monolíticos tradicionales presentan limitaciones en:

- Escalabilidad horizontal
- Tolerancia a fallos
- Independencia de despliegue
- Evolución tecnológica

Ante este contexto, se plantea la necesidad de diseñar un sistema bancario basado en una arquitectura de microservicios que adopte principios de **DevOps y Site Reliability Engineering (SRE)**, priorizando la confiabilidad, el desacoplamiento y la comunicación asíncrona entre componentes.

Definición de Microservicios

Visión general del dominio
El sistema bancario estará compuesto por los siguientes microservicios:

1. **Customer Service**
2. **Account Service**
3. **Transaction Service**

4. **Payment Service**
5. **Notification & Audit Service**

Customer Service
Responsabilidad
Gestión de clientes del sistema bancario.
Funciones

- Registro de cliente
- Validación de identidad (foto del documento de identificación)
- Actualización de datos

Datos

- customerId
- nombre (completo)
- documento
- correo
- Fecha de nacimiento (validar coherencia del campo)
- Dirección
- Username (generado a partir del nombre del usuario)

Notas SRE

- Manejo de consistencia eventual
- Protección contra condiciones de carrera
- Idempotencia en actualizaciones

Account Service
Responsabilidad
Gestión de cuentas bancarias y balances.
Funciones

- Creación de cuenta
- Consulta de saldo
- Actualización de balance
- Proceso automático de desactivación de cuentas inactivas con balance menor a Q50.00 tras 6 meses de inactividad.

Datos

- accountId
- customerId
- balance
- estado

Notas SRE

- Uso de **correlationId**
- Manejo de retries

- Garantía de no duplicidad

Transaction Service

Responsabilidad

Orquestación lógica de transacciones financieras.

Funciones

- Transferencias entre cuentas
- Validación de fondos
- Registro de transacciones

Datos

- transactionId
- sourceAccount
- targetAccount
- amount
- status

Notas SRE

- Uso de **correlationId**
- Manejo de retries
- Garantía de no duplicidad

Payment Service

Responsabilidad

Procesamiento de pagos externos o internos.

Funciones

- Procesar pagos
- Validar operaciones financieras
- Integración futura con sistemas externos

Datos

- paymentId
- transactionId
- status

Notas SRE

- Separación de lógica de pagos
- Manejo de fallos externos
- Simulación de sistemas bancarios reales

Notification & Audit Service

Responsabilidad

Registro de eventos y notificaciones.

Funciones

- Envío de notificaciones
- Auditoría del sistema
- Logging distribuido (base conceptual para observabilidad)

Datos

- eventId
- tipoEvento
- timestamp

- payload

Notas SRE

- Base para observabilidad futura
- No afecta lógica de negocio
- Altamente desacoplado

Reglas de diseño obligatorias

- Cada microservicio:
  - Tiene su propia base de datos
  - No accede a la base de otro servicio
  - Se comunica solo por eventos
- No existe:
  - Llamadas HTTP entre servicios
  - Acoplamiento directo

Patrones esperados

Los estudiantes pueden aplicar:

- Event-Driven Architecture
- Choreography (recomendada)
- Request/Reply async
- Saga (obligatorio)

Consideraciones SRE (obligatorias)

Cada servicio debe considerar:

- Idempotencia
- Retries
- Manejo de errores
- Eventual consistency
- Trazabilidad (correlationId)

Observaciones críticas

Se penaliza:

- Mezclar responsabilidades entre servicios
- Servicios “god object”
- Comunicación directa entre servicios
- Falta de eventos claros
- Falta de modelo de dominio

4.2 Alcance del proyecto

Alcance obligatorio:

El proyecto consiste en el diseño e implementación de un sistema bancario distribuido que cumpla con los siguientes requerimientos:

- Implementación de **5 microservicios** con responsabilidades claramente definidas dentro del dominio bancario
- Uso de un **API Gateway** como punto de entrada al sistema
- Implementación de un modelo de **comunicación completamente asíncrono entre microservicios**
- Uso de un mecanismo de mensajería, el cual puede ser seleccionado por el equipo:
    - Kafka
    - RabbitMQ
    - NATS
    - gRPC (en modalidad asíncrona)
- Despliegue del sistema en un **cluster local de Kubernetes**
- Uso de **bases de datos externas al cluster** (contenedores Docker o servicios locales)
- Aplicación de principios de **DevOps y SRE desde el diseño**

Además, el proyecto debe incluir la siguiente documentación:

- Diagramas de arquitectura
- Diagramas de despliegue
- Diagramas de secuencia
- Casos de uso
- Definición de:
    - SLI (Service Level Indicators)
    - SLO (Service Level Objectives)
    - SLA (Service Level Agreements)
- Diagramas Obligatorios y Criterios de Evaluación (C4 + UML)

Diagramas Obligatorios y Criterios de Evaluación (C4 + UML)

Objetivo

Establecer los lineamientos para la elaboración de diagramas arquitectónicos y de comportamiento del sistema, utilizando modelos **C4** y **UML**, con el fin de evidenciar comprensión de:

- Arquitectura de microservicios
- Comunicación asíncrona
- Desacoplamiento
- Diseño orientado a confiabilidad (SRE)

Diagramas obligatorios

1. Diagrama de Contexto (C4 - Nivel 1)

Descripción

Representa el sistema como una caja negra y su interacción con actores externos.

Debe incluir

- Usuario(s)
- Sistema bancario
- API Gateway
- Sistemas externos (si aplica)
- Tipo de interacción (debe indicar asincronía si aplica)

Criterios de evaluación

- Claridad en los límites del sistema
- Identificación correcta de actores
- Coherencia con el dominio bancario
- Representación correcta del Gateway como punto de entrada

Diagrama de Contenedores (C4 - Nivel 2)

Descripción

Muestra la estructura interna del sistema en términos de contenedores (microservicios, gateway, broker, bases de datos).

Debe incluir

- Los 5 microservicios
- API Gateway
- Broker de mensajería
- Bases de datos (externas)
- Relaciones entre componentes
- Tipo de comunicación:
    - Asíncrona (obligatorio entre microservicios)

Criterios de evaluación

- Separación clara de responsabilidades
- Uso correcto de comunicación asíncrona
- Ausencia de acoplamiento indebido
- Correcta ubicación del broker en la arquitectura
- Independencia de bases de datos por servicio

Diagrama de Componentes (C4 - Nivel 3)

Descripción

Descompone al menos un microservicio en sus componentes internos.

Debe incluir

- Controladores / Handlers
- Servicios de dominio
- Adaptadores (ej: publisher/subscriber)
- Repositorios
- Integración con el broker

Criterios de evaluación

- Separación de capas (clean architecture o similar)
- Manejo adecuado de eventos
- Bajo acoplamiento interno

- Coherencia con principios de diseño

Diagrama de Despliegue (UML)

Descripción

Representa la distribución física del sistema en infraestructura.

Debe incluir

- Cluster de Kubernetes (local)
- Pods por microservicio
- API Gateway desplegado
- Broker desplegado (o externo)
- Bases de datos externas (fuera del cluster)
- Nodos (si aplica)

Criterios de evaluación

- Correcta representación del entorno Kubernetes
- Separación entre cluster y servicios externos
- Ubicación adecuada de componentes
- Viabilidad técnica del despliegue

Diagramas de Secuencia (UML)

Descripción

Modelan la interacción entre componentes para casos de uso clave.

Debe incluir (mínimo 2 casos):

- Transferencia bancaria
- Creación de cuenta

Debe reflejar

- Flujo asíncrono
- Publicación y consumo de eventos
- Uso del broker

- Correlación de mensajes (si aplica)

Criterios de evaluación

- Correcto uso de asincronía
- Representación clara de eventos
- Secuencia lógica del flujo
- Manejo de interacción distribuida

Diagrama de Casos de Uso (UML)

Descripción

Define las funcionalidades del sistema desde la perspectiva del usuario.

Debe incluir

- Actores
- Casos de uso principales:
    - Crear cuenta
    - Consultar saldo
    - Transferir dinero
- Relación entre casos

Criterios de evaluación

- Cobertura funcional adecuada
- Claridad en los actores
- Coherencia con el dominio

Requisitos transversales (aplican a todos los diagramas)

Todos los diagramas deben:

- Ser consistentes entre sí
- Reflejar comunicación **asíncrona obligatoria**
- Mantener separación de responsabilidades
- Estar correctamente etiquetados (tecnología, protocolo, tipo de comunicación)

- Ser legibles y comprensibles

Restricciones arquitectónicas

- No se permite comunicación síncrona (HTTP u otros protocolos similares) entre microservicios
- Toda interacción entre servicios debe realizarse mediante mecanismos asíncronos
- El API Gateway debe actuar únicamente como punto de entrada al sistema
- Cada microservicio debe ser autónomo y desacoplado.
- Cada microservicio debe de poseer su base de datos.
- No se permite antipatrones de microservicios. Tales como microservicios conectados a múltiples bases de datos. La forma de comunicación debe ser estrictamente entre microservicios, ningún microservicio puede consultar bases de datos que no sean la correspondiente. Caer en el incumplimiento de esta restricción será penalizable con el 50% de la nota obtenida.

Flexibilidad de diseño

El equipo tiene la libertad de:

- Seleccionar el lenguaje de programación:
  - Java
  - TypeScript
  - Go
- Seleccionar frameworks:
  - Spring Boot 4
  - NestJS
- Definir el patrón de comunicación:
  - Request/Reply asíncrono
  - Publicación/Suscripción (Pub/Sub)
  - Arquitectura basada en eventos (Event-Driven)
  - Modelo híbrido

4.3 Recursos y herramientas a utilizar

Tipo	Categoría (Software / hardware / Plataforma / Etc)	Descripción
Obligatorio	Lenguaje	Java, TypeScript o Go
Obligatorio	Framework	Spring Boot 4 o NestJS
Obligatorio	Orquestación	Kubernetes (local)
Obligatorio	Contenedores	Docker
Obligatorio	Mensajería	Kafka, RabbitMQ, NATS o gRPC async
Obligatorio	API Gateway	Implementado según stack seleccionado
Obligatorio	Base de datos	Externa al cluster

4.4 Entregables

Tipo	**Descripción**
Documentación	• Documento técnico completo del sistema • Diagramas arquitectónicos • Definición de SLI, SLO y SLA • Casos de uso detallados • Justificación de decisiones arquitectónicas • En formato markdown • Manual de usuario
Repositorio del Proyecto	Nombre del Repositorio: **Proyecto-SA-<<SECCIÓN>>-<<#Grupo>>**

	Debebe de utilizar estrategias de monorepo.
Código fuente	Código fuente separado por microservicio
K8s	Archivos de configuración para despliegue en Kubernetes
Frontend	Deberá diseñar una interfaz intuitiva, agradable y funcional para la validación de las funcionalidades de cada microservicio

5. Material de apoyo

Categoría	Link	Descripción
Documentación oficial	https://kubernetes.io/docs/	Documentación oficial de Kubernetes para despliegue, pods, servicios y configuración del cluster
Documentación oficial	https://docs.docker.com/	Guía oficial de Docker para contenedorización de microservicios
Documentación oficial	https://spring.io/projects/spring-boot	Framework backend para Java orientado a microservicios
Documentación oficial	https://docs.nestjs.com/	Framework backend en TypeScript para construcción de servicios escalables
Documentación oficial	https://kafka.apache.org/documentation/	Documentación de Apache Kafka para sistemas distribuidos basados en eventos
Documentación oficial	https://www.rabbitmq.com/documentation.html	Documentación de RabbitMQ para mensajería asíncrona basada en colas

Documentación oficial	https://docs.nats.io/	Documentación de NATS para comunicación ligera y de alto rendimiento
Documentación oficial	https://grpc.io/docs/	Documentación de gRPC, incluyendo patrones de comunicación
Manual	https://martinfowler.com/articles/microservices.html	Conceptos fundamentales de arquitectura de microservicios
Manual	https://microservices.io/patterns/index.html	Catálogo de patrones de microservicios (event-driven, saga, etc.)
Manual	https://microservices.io/patterns/data/saga.html	Explicación detallada del patrón Saga para transacciones distribuidas
Manual	https://c4model.com/	Modelo C4 para documentación de arquitectura de software
Manual	https://www.uml.org/	Introducción a UML y tipos de diagramas
Documentación oficial	https://sre.google/sre-book/table-of-contents/	Libro oficial de Site Reliability Engineering (Google)
Manual	https://sre.google/workbook/table-of-contents/	Guía práctica de SRE (SLI, SLO, SLA, confiabilidad)
Ejemplo	https://github.com/eventuate-tram/eventuate-tram-sagas-examples-customers-and-orders	Ejemplo de implementación de Saga en microservicios
Manual	https://12factor.net/	Principios para diseño de aplicaciones modernas y escalables
Documentación oficial	https://helm.sh/docs/	(Opcional) Gestión de despliegues en Kubernetes
Manual	https://aws.amazon.com/what-is/event-driven-architecture/	Introducción clara a arquitectura orientada a eventos

6. Metodología

A continuación se presenta una metodología estructurada que el estudiante debe seguir para abordar el desarrollo del proyecto, priorizando el diseño arquitectónico, la comunicación asíncrona y la confiabilidad del sistema.

Pasos a seguir:

1. Paso 1: Definición del dominio
   Definir los 5 microservicios, sus responsabilidades, datos y eventos. Asegurar que cada servicio sea autónomo y no comparta base de datos con otros.

2. Paso 2: Diseño de microservicios
   Analizar el dominio bancario y definir las entidades principales (cliente, cuenta, transacción, pago). Identificar límites de contexto (bounded contexts) para dividir correctamente los microservicios.

3. Paso 3: Definición del modelo de comunicación
   Seleccionar el mecanismo de mensajería (Kafka, RabbitMQ, NATS o gRPC asíncrono) y definir el patrón de comunicación:
   - Event-driven
   - Request/Reply
   - Pub/Sub

   Justificar la elección en términos de desacoplamiento y resiliencia.

4. Paso 4: Diseño de eventos y contratos
   Definir los eventos que se intercambiarán entre servicios:
   - Nombre del evento
   - Estructura del payload
   - Uso de correlationId

   Establecer convenciones para topics o subjects.

5. Paso 5: Diseño de la arquitectura (Diagramas)
   Elaborar los diagramas obligatorios:
   - C4 (Contexto, Contenedores, Componentes)
   - UML (Secuencia, Despliegue, Casos de uso)

   Asegurar consistencia entre todos los diagramas.

6. Paso 6: Definición del flujo de negocio (Saga)
   Diseñar el flujo de transferencia bancaria utilizando el patrón Saga:
   - Flujo exitoso
   - Manejo de fallos
   - Eventos de compensación

7. Paso 7: Definición de métricas SRE
   Definir:
   - SLI (indicadores)

- SLO (objetivos)
- SLA (acuerdos)

Relacionarlos con el comportamiento del sistema.

8. Paso 8: Implementación de microservicios
   Desarrollar los microservicios utilizando el lenguaje y framework seleccionado.
   Implementar:
   - Publicación de eventos
   - Consumo de eventos
   - Lógica de negocio

9. Paso 9: Implementación del API Gateway
   Desarrollar el API Gateway como punto de entrada del sistema. Este debe integrarse con el mecanismo de mensajería para iniciar flujos de negocio de manera asíncrona.

10. Paso 10: Contenerización del sistema
    Crear Dockerfiles para cada microservicio y el gateway. Validar ejecución local de cada componente.

11. Paso 11: Despliegue en Kubernetes
    Configurar el cluster local (minikube, kind o k3d) y desplegar:
    - Microservicios
    - Gateway
    - Broker (si aplica)

    Las bases de datos deben estar fuera del cluster.

12. Paso 12: Pruebas de integración
    Validar el flujo completo del sistema:
    - Transferencias exitosas
    - Manejo de errores
    - Consumo correcto de eventos
13. Paso 13: Documentación final
    Integrar toda la documentación:
    - Diagramas
    - Justificación tecnológica
    - SLI/SLO/SLA
    - Casos de uso

Recomendaciones

- Diseñar antes de programar; evitar comenzar directamente con código
- Mantener separación estricta entre microservicios
- No utilizar comunicación síncrona entre servicios
- Definir claramente los eventos antes de implementar
- Implementar idempotencia en consumidores de eventos
- Utilizar identificadores de correlación (correlationId) en todos los flujos

- Validar escenarios de fallo, no solo casos exitosos
- Evitar soluciones monolíticas disfrazadas de microservicios
- Justificar todas las decisiones arquitectónicas
- Mantener consistencia entre diagramas y código

7. Cronograma

Tipo	Fecha Inicio	Fecha Fin
Asignación de proyecto	2026-08-06	2026-08-06
Elaboración	2026-08-06	2026-09-03
Calificación	2026-09-05	2026-09-05

8. Rúbrica de Calificación

8.1 Requisitos para optar a la calificación

Tema	**Descripción**	**Cumple (Sí/No)**
Arquitectura definida	El proyecto presenta una arquitectura de microservicios claramente documentada	
Número de microservicios	Se implementan exactamente **5 microservicios** con responsabilidades diferenciadas	
API Gateway	Se implementa un API Gateway funcional como punto de entrada al sistema	
Comunicación asíncrona Uso de broker	Toda la comunicación entre microservicios es **100% asíncrona** Se utiliza correctamente un mecanismo de mensajería (Kafka, RabbitMQ, NATS o gRPC async)	

Desacoplamiento	No existe comunicación directa (HTTP u otra síncrona) entre microservicio		
Base de datos por servicio	Cada microservicio posee su propia base de datos independiente		
Bases de datos externas	Las bases de datos se encuentran fuera del cluster de Kubernetes		
Despliegue en Kubernetes	El sistema está desplegado en un cluster local funcional		
Contenerización	Todos los servicios están correctamente dockerizados		
Eventos definidos	El sistema define claramente eventos (nombres, payloads, flujo)		
Uso de correlationId	Se implementa trazabilidad mediante identificadores de correlación		
Idempotencia	Los servicios manejan correctamente duplicidad de eventos		
Manejo de errores	Se contemplan escenarios de fallo en el flujo (no solo happy path)		
Implementación de Saga	Se implementa un flujo distribuido basado en Saga (coreografía)		
Diagramas completos	Se entregan todos los diagramas requeridos (C4 + UML)		
Consistencia de diagramas	Los diagramas son coherentes entre sí y con la implementación		

Casos de uso	Se presentan casos de uso correctamente definidos	
Definición SRE	Se incluyen SLI, SLO y SLA definidos correctamente	
Justificación técnica	Se justifica la elección de tecnologías y decisiones arquitectónicas	
Código funcional	El sistema ejecuta correctamente el flujo de transferencia bancaria	

8.2 Detalle de la Calificación

	Criterio/Nivel			**Nivel de evaluación**		
No.	**Criterio de evaluación**	**Punteo maximo**		**Satisfactorio 100% - 61%**	**Necesita mejorar 60% - 0%**	**Punteo Obtenido**
1	Habilidades	40				
1.1	Trabajo en equipo	10	Descripción	Evidente distribución de responsabilidades, integración coherente de servicios. Respaldado con Commits y metodología ágil.	Trabajo desbalanceado o sin integración clara. No commits necesarios para validar trabajo en equipo y sin evidencia de uso de metodología ágil.	
			Rango del punteo	10-6	5-0	
1.2	Documentación técnica	10	Descripción	Clara, estructurada, consistente con la implementación	Incompleta, inconsistente o superficial	
			Rango del punteo	10-6	5-0	

1.3	Justificación de decisiones	10	Descripción	Decisiones bien argumentadas con trade-offs técnicos	Sin justificación o basada en opiniones	
			Rango del punteo	10-6	5-0	
1.4	Claridad en diagramas	10	Descripción	Diagramas legibles, coherentes y bien estructurados	Diagramas confusos o incorrectos	
			Rango del punteo	10-6	5-0	
	Sub-Total de Puntos	40				

2	Conocimientos	60				
2.1	Separación de servicios	5	Descripción	Servicios bien definidos y desacoplados	Servicios mal definidos o acoplados	
			Rango del punteo	5-3	2-0	
2.2	Diseño de dominio	5	Descripción	Modelo coherente con dominio bancario	Modelo pobre o inconsistente	
			Rango del punteo	5-3	2-0	
2.3	Independencia de datos	5	Descripción	Base de datos por servicio	DB compartida o acoplamiento	
			Rango del punteo	5	0	
2.4	Uso correcto del Gateway	5	Descripción	Gateway como entrypoint, sin lógica de negocio	Gateway mal utilizado (orquestador)	
			Rango del punteo	5	0	

2.5	Uso de broker	5	Descripción	Uso correcto y consistente	Uso incorrecto o superficial	
			Rango del punteo	5	0	
2.6	Desacoplamiento	5	Descripción	Servicios independientes mediante eventos	Dependencias directas o acoplamiento	
			Rango del punteo	5-3	2-0	
2.7	Flujo distribuido	5	Descripción	Flujo correctamente modelado	Flujo incompleto o incorrecto	
			Rango del punteo	5	0	
2.8	Manejo de fallos	5	Descripción	Compensaciones y errores definidos	No se manejan fallos	
			Rango del punteo	5	0	

2.9	Definición de métricas	5	Descripción	SLI, SLO y SLA claros y medibles	Definiciones vagas o incorrectas	
			Rango del punteo	5	0	
2.10	Despliegue funcional	5	Descripción	Sistema correctamente desplegado en cluster	Fallos en despliegue o configuración o no uso de K8s	
			Rango del punteo	5	0	
	Sub-Total de Puntos	60				
	Total	**100**				

8.3 Comentarios Generales

Se reducirá significativamente la nota si se detecta:

- Comunicación síncrona entre microservicios 50%
- Uso de una sola base de datos compartida 100%
- Falta de asincronía real 50%
- Arquitectura monolítica disfrazada 100%
- Inconsistencia entre diagramas y código 25%
- No despliegue en Kubernetes (local) 100%